Deterministic PII Tokenization for LLM API Traffic: Test Suite Results and Empirical Analysis of the NOπI Redaction Engine

Michael Roling, CISSP and Tony Alex, Ph.D.
EnigmaVault.io
March 23, 2026

Abstract

PII protection for LLM API traffic is typically validated by inspecting the response returned to the calling application. This is methodologically wrong: if the proxy restores original values before returning the response, as it should, then checking the response will always show PII and produce false failures. This paper presents 109 automated tests executed against the NOπI PII redaction engine, a deterministic tokenization proxy that intercepts LLM API traffic and replaces sensitive entities with secure tokens before they reach the model provider. The test suite uses Langfuse trace validation to verify PII handling at the LLM input span, the correct architectural layer, correcting this flaw in the prior version. Results demonstrate an 89% overall pass rate (95% effective rate excluding design limitations and test bugs) across eight domains, with 100% pass rates in healthcare (17 tests), legal (8 tests), and substring integrity (35 tests). Four unexpected passes reveal detection capabilities beyond documented expectations, including CJK character handling and obfuscated email recognition. Of five nominal failures, only three represent actual engine limitations; the remaining two are a known design property and a test bug. The v1-to-v2 comparison shows a 35 percentage point improvement driven primarily by correcting the test validation methodology rather than by changes to the engine itself.

I. INTRODUCTION

The enterprise adoption of large language models has introduced a measurable category of data exposure risk. Nasr et al. [1] demonstrated that adversaries can extract gigabytes of memorized training data from production language models, including ChatGPT, using divergence attacks that cause aligned models to emit training data at 150 times their normal rate. Of the generated responses tested, 16.9% contained memorized PII with 85.8% of that PII confirmed as authentic. These findings establish that data sent to LLM providers is not merely processed and discarded; it can be memorized and subsequently extracted.

The scale of enterprise exposure compounds this risk. LayerX Security's 2025 Enterprise AI and SaaS Data Security Report [2] found that 77% of enterprise employees paste data into generative AI prompts, and nearly 40% of files uploaded to these tools contain PII or payment card industry (PCI) data. Lasso Security's 2025 research [3] corroborates this, finding that 13% of all enterprise GenAI prompts contain sensitive organizational data, with 11.2% of prompts containing personal data flagged for email addresses and payment information. For organizations bound by HIPAA, GDPR, PCI-DSS, or SOX, this volume of uncontrolled PII transmission to third-party model infrastructure creates ongoing compliance violations.

Existing mitigations have proven insufficient. Lukas et al. [4] evaluated PII scrubbing and differential privacy defenses on fine-tuned GPT-2 models across healthcare, legal, and email domains. Their findings showed that even sentence-level differential privacy still leaks approximately 3% of PII sequences, and that novel extraction attacks can recover up to 10 times more PII than previously demonstrated methods. Scrubbing reduces but does not eliminate PII exposure, and it operates at the cost of dataset utility.

NOπI takes a different approach. Rather than scrubbing PII from training data or relying on provider-side data handling agreements, NOπI operates as a deterministic tokenization proxy for LLM API traffic. The architecture is described in Section II.

This paper presents the results of 109 automated tests executed against the NOπI redaction engine on March 23, 2026. The tests span eight domains: healthcare, legal, financial services, developer workflows, streaming, multi-turn consistency, substring integrity, and adversarial edge cases. The test suite uses Langfuse trace validation to

All Rights Reserved NOπI 2026www.nopii.co

inspect the LLM input span, ensuring that PII protection is verified at the correct architectural layer.

II. ARCHITECTURE AND TEST ENVIRONMENT

A. Proxy Architecture

NOπI operates as a reverse proxy positioned between the calling application and the LLM provider. The processing pipeline executes in four stages:

1. Intercept. The proxy receives the outbound API request from the calling application.

2. Detect and tokenize. The NER engine identifies PII entities in the prompt and replaces each with a deterministic token. The same input string always produces the same token, preserving referential consistency within and across conversation turns.

3. Forward. The sanitized prompt is forwarded to the model provider. The provider processes a prompt containing no real PII.

4. Restore. The proxy intercepts the model's response, replaces tokens with the original values, and returns the fully usable response to the calling application.

This architecture means the model provider never processes real PII. The calling application receives a response indistinguishable from one produced without the proxy.

B. Test Environment

All tests were executed against the NOπI proxy endpoint (api.test.nopii.co/v1) routing to OpenAI's GPT-4o model. The test suite version is v2, which introduces Langfuse-based trace validation as the primary assertion mechanism.

The validation approach inspects the llm-call span recorded in Langfuse, which captures the exact payload delivered to the LLM after NOπI has performed tokenization. This corrects a fundamental flaw in the v1 suite, which checked the de-tokenized response returned to the client. Since de-tokenization is designed to restore original PII for application use, v1 tests produced false failures: they detected PII that had been correctly tokenized at the LLM layer but was intentionally restored for the application. The v2 methodology validates what the model actually sees.

III. RESULTS

Of 109 tests, 97 passed, 5 produced nominal failures, 3 failed as expected (XFAIL), and 4 passed unexpectedly (XPASS). The overall pass rate is 89%. However, two of the five nominal failures are not engine defects: one (MT-001) is an inherent property of deterministic tokenization, and one (SC-001) is a test bug with severity "None." Excluding these, the effective pass rate on tests where the engine is expected to succeed is 97 of 102, or 95%. Table I summarizes the aggregate results.

TABLE I: AGGREGATE TEST RESULTS

Status	Count
Passed	97
Failed (engine)	3
Failed (design limit)	1
Failed (test bug)	1
XFAIL (expected)	3
XPASS (unexpected pass)	4
Total	**109**

Table II presents the pass rate by domain. Three domains achieved 100% pass rates: Healthcare, Legal, and Substring Integrity. The following subsections analyze each domain.

TABLE II: DOMAIN SCORECARD

Domain	Pass	Fail	XFail	XPass	Score	Eff.
Healthcare	17	0	0	0	100%	100%
Legal	8	0	0	0	100%	100%
Substring	35	0	0	0	100%	100%
Developer	11	0	1	0	100%	100%
Financial	11	1	0	0	92%	92%
Streaming	5	1*	0	0	83%	100%
Multi-turn	3	1	0	0	75%	100%
Edge Cases	6	2	2	4	71%	75%
Overall	**97**	**5**	**3**	**4**	**89%**	**95%**

** Test bug (SC-001). Eff. = effective score excluding design limits and test bugs.*

A. Healthcare (17/17, 100%)

The healthcare domain tested five clinical scenarios: patient intake, multi-party referrals, laboratory results, mental health progress notes, and electronic prescriptions. All 17 tests passed. These scenarios represent the core PHI exposure vectors in clinical LLM workflows.

The mental health test (HC-004) warrants specific attention. The prompt contained suicidal ideation, substance use diagnoses, employer details, and divorce proceedings, a density of sensitive information typical of behavioral health documentation and precisely the kind of content that creates HIPAA liability when transmitted to LLM providers. All three person names were tokenized. Clinical terminology (Metformin, ICD-10 codes, dosages) was preserved in the LLM output, confirming that tokenization does not degrade clinical reasoning.

B. Legal (8/8, 100%)

The legal suite tested contract clauses, deposition transcripts, and immigration filings. These scenarios contain

All Rights Reserved NOπI 2026

entity types that are notoriously difficult for NER systems: mixed personal and corporate names, natural language date formats, and case numbers that resemble other numeric identifiers.

A date of birth written as "January 19th, 1978" was correctly tokenized, demonstrating detection of natural language date formats beyond the standard MM/DD/YYYY pattern. Company names were preserved and not falsely flagged as person names, a common false positive in legal text where firm names often follow patterns similar to personal names.

C. Financial Services (11/12, 92%)

11 of 12 tests passed. The suite covered credit card disputes, wire transfers, mortgage applications, international IBAN payments, and U.S. tax filings. These scenarios involve data categories protected under PCI-DSS, including primary account numbers, cardholder names, and bank account identifiers. Unicode names and international address formats were correctly handled. SSNs were correctly tokenized across all scenarios where they appeared.

The single failure (FN-003) is the most compliance-significant engine limitation identified in this test suite. In a mortgage application, short first names ("Daniel," "Lisa") paired with the surname "Okafor" were not recognized as person entities. In a mortgage context, applicant names are protected under the Gramm-Leach-Bliley Act (GLBA) and Regulation P. Failure to tokenize applicant names in a loan document represents a potential compliance event. The root cause is insufficient contextual signal for short, common first names in the NER pipeline. The planned remediation, field-label proximity hinting, will use structural cues from the surrounding document to boost entity confidence for names adjacent to known PII fields. This is prioritized for the next engine release.

D. Developer Workflows (11/13, 100% effective)

11 of 13 tests passed. The two non-passes are an expected failure (API keys and credentials, which are not PII and fall outside the engine's scope) and a design-correct result. The suite verified detection in customer support prompts, email summarization, code generation with real credentials, RAG context injection with structured JSON, and translation prompts with international PII formats including Irish PPS numbers.

E. Streaming (5/6, 83%; 100% effective)

The streaming suite tests PII that is split across server-sent event (SSE) chunk boundaries, a technically challenging scenario because entity detection must operate across partial payloads. Five of six tests passed. The single failure (SC-001) is a test design error: the assertion checked the de-tokenized response rather than the LLM input span, replicating the v1 methodology flaw in one remaining test. The failure is classified as severity "None" with a recommendation to rewrite the assertion. No engine defect is indicated.

F. Multi-turn Consistency (3/4, 75%; by-design limitation)

Three tests confirmed that the same entity receives the same deterministic token across multiple conversation turns, and that PII remains tokenized in subsequent turns. The single failure (MT-001) tested whether two different individuals named "John Smith" would receive different tokens. They did not. This is an inherent property of deterministic tokenization: the same input string produces the same output. Context-aware entity disambiguation would require session-level identity resolution and is outside the current tokenization model's scope. This test should be reclassified as XFAIL.

G. Edge Cases (6/14, 71%)

The edge case domain is designed to push the engine beyond documented capabilities. Of 14 tests, 6 passed, 2 produced real engine failures, 2 expected failures, and 4 unexpected passes. The two real failures are analyzed in Section VII.

IV. UNEXPECTED PASSES

Four tests that were expected to fail instead passed, revealing detection capabilities beyond documented specifications:

Obfuscated email. The string "john dot doe at gmail dot com" was tokenized despite being written in natural language rather than standard email format.

Spaced credit card. A card number entered as individual digits with spaces ("4 5 3 2 7 5 6 2 9 1 0 2 3 4 5 6") was detected and tokenized.

CJK name detection. The Japanese kanji name 田中太郎 was tokenized, confirming non-Latin character set handling.

Japanese date format. The date 1990年4月15日 was detected and tokenized.

These results suggest the detection engine incorporates contextual analysis beyond pattern matching. The corresponding test expectations should be updated from XFAIL to standard pass criteria, and CJK and obfuscated format handling should be added to the product documentation as supported features.

V. SUBSTRING INTEGRITY

A common failure mode in PII tokenization is substring corruption: a naive system detects a PII-like pattern inside a longer word and mangles the output. NOπI was tested against 35 collision-prone words across four categories: date-like substrings (update, validate, candidate), name-like substrings (filename, username, namespace), phone-like substrings (microphone, telephone, smartphone), and other collision patterns (allocation, relocation, headdress). All 35 tests passed, confirming that the tokenization engine operates at entity boundaries and does not corrupt legitimate vocabulary.

All Rights Reserved NOπI 2026

VI. TEST SUITE EVOLUTION: V1 TO V2

The v2 test suite improved the pass rate from 54% to 89%, a gain of 35 percentage points. The critical finding is that this improvement was driven almost entirely by correcting the test methodology, not by changes to the engine. Table III summarizes the changes.

TABLE III: V1 TO V2 COMPARISON

Category	v1	v2	Change
PII leaks	~20 FAIL	23P / 2F	Correct layer
Token types	~18 FAIL	18 PASS	Visible in input
False pos.	3 PASS	2P / 1F	Stricter check
Multi-turn	4 FAIL	3P / 1 limit	Consistency OK
Streaming	~3 FAIL	5P / 1 bug	Verified
Substring	35 PASS	35 PASS	No change
CJK	1XF / 1F	2 XPASS	Working
Total	**54%**	**89%**	**+35 pts**

This result has implications beyond NOπI. Any PII protection system that operates as a proxy, tokenizing on the way in and restoring on the way out, will produce false failures if tested at the application layer. The v1 experience suggests that other PII protection solutions may be reporting artificially low efficacy for the same reason.

VII. FAILURE ANALYSIS

Five nominal failures were identified. We classify them into three categories: engine limitations (3), design properties (1), and test bugs (1).

A. Engine Limitations

EC-002: Ambiguous name false positives. Common English words "Grace," "Bill," and "Amber" were flagged as person names in non-PII context. The underlying NER model lacks disambiguation for words that function as both common nouns and proper names. Remediation: implement context-aware allow-lists with confidence thresholds that suppress entity classification when surrounding context does not support a person-name interpretation. **Severity: Medium.**

EC-005: SSN in code comments. An SSN embedded inside a Python code comment was not tokenized. The detection engine does not currently scan inside code block delimiters. This is a meaningful gap: developer workflows are a primary use case for NOπI, and developers routinely paste code containing real customer data into LLM prompts for debugging assistance. The threat model for this failure is not edge-case; it is core to the developer persona. Remediation: extend scanning to code blocks, string literals, and inline comments. **Severity: Medium.**

FN-003: Mortgage application names. Short first names "Daniel" and "Lisa" were not recognized as person entities alongside the surname "Okafor" in a mortgage application. This is the most compliance-significant failure in the suite, as discussed in Section III-C. Remediation: field-label proximity hinting, prioritized for the next engine release. **Severity: High.**

B. Design Property

MT-001: Same-name disambiguation. Two different individuals named "John Smith" received identical tokens. This is an inherent property of deterministic tokenization: the same input string produces the same output. Context-aware disambiguation would require session-level identity resolution, which is architecturally distinct from string-level tokenization. This test should be reclassified as XFAIL. **Severity: Low (by design).**

C. Test Bug

SC-001: Streaming validation layer. The test assertion checked the de-tokenized response rather than the LLM input span, replicating the v1 methodology flaw. This is not an engine failure. Remediation: rewrite assertion to use Langfuse trace validation. **Severity: None.**

VIII. CONCLUSION

The NOπI PII redaction engine demonstrates production-grade performance across regulated domains. The 100% pass rates in healthcare and legal testing indicate reliability for the highest-stakes compliance scenarios. Of the three real engine limitations identified, none represent fundamental architectural weaknesses: two are NER confidence issues addressable through allow-lists and contextual hinting, and one is a scanning scope gap with a clear remediation path.

The most significant finding is methodological. The 35 percentage point improvement from v1 to v2 was driven by correcting the test validation layer, not by engine changes. Any PII protection proxy that tokenizes on the way in and restores on the way out will produce false failures if tested at the application response layer. Organizations evaluating PII protection for LLM traffic, whether building their own or assessing vendors, should validate at the LLM input span. Testing at any other layer conflates the proxy's intentional restoration behavior with a failure to protect.

The four unexpected passes in CJK, obfuscated email, and non-standard format detection suggest capabilities that exceed the documented specification. These should be formalized as supported features and reflected in updated product documentation.

The FN-003 failure (mortgage application names) is prioritized for remediation as the most compliance-significant gap. The field-label proximity hinting approach is expected to address this class of short-name detection failures across financial services documents without introducing false positives in other domains.

IX. REFERENCES

All Rights Reserved NOπI 2026

[1] M. Nasr, N. Carlini, J. Hayase, M. Jagielski, A. F. Cooper, D. Ippolito, C. A. Choquette-Choo, E. Wallace, F. Tramer, and K. Lee, "Scalable Extraction of Training Data from (Production) Language Models," arXiv:2311.17035, November 2023.

[2] LayerX Security, "Enterprise AI and SaaS Data Security Report 2025," LayerX Security, 2025. Telemetry data collected through enterprise browser monitoring across global organizations.

[3] Lasso Security, "Lasso Research Reveals 13% of Generative AI Prompts Contain Sensitive Organizational Data," Lasso Security Research, 2025. Based on data collected between December 2023 and February 2025.

[4] N. Lukas, A. Salem, R. Sim, S. Tople, L. Wutschitz, and S. Zanella-Beguelin, "Analyzing Leakage of Personally Identifiable Information in Language Models," in 2023 IEEE Symposium on Security and Privacy (SP), San Francisco, CA, USA, 2023, pp. 346-363, doi: 10.1109/SP46215.2023.00154.

All Rights Reserved NOπI 2026